

Deep Learning LSTM

Dr. SHIV SHANKAR PRASAD SHUKLA

Assistant Professor (SCAI)

VIT BHOPAL UNIVERSITY

Artificial intelligence in detail

- As the data sets get more complex, analysts needs some standard tool.
- AI : simulation of human intelligence in machine
- Machine learning:

“said to learn from experience with respect to some class of tasks a performance measure by P. If performance at tasks in the class as measure by P improves with the experience.”

- Tom Mitchell

Machine learning in brief

- Supervised learning : data is in structured way and the historical data contain the features as well as the label.
- Unsupervised learning : data is in unstructured way , thus there is no label
- Re-inforcement learning : to control the model or system if system give the optimize correct result give some Reward otherwise penalty.

Deep learning

- Subset of Machine Learning
- Uses Neural Networks with multiple layers
- Learns hierarchical features
- Used in NLP, Image Processing, Speech Recognition, Time-Series

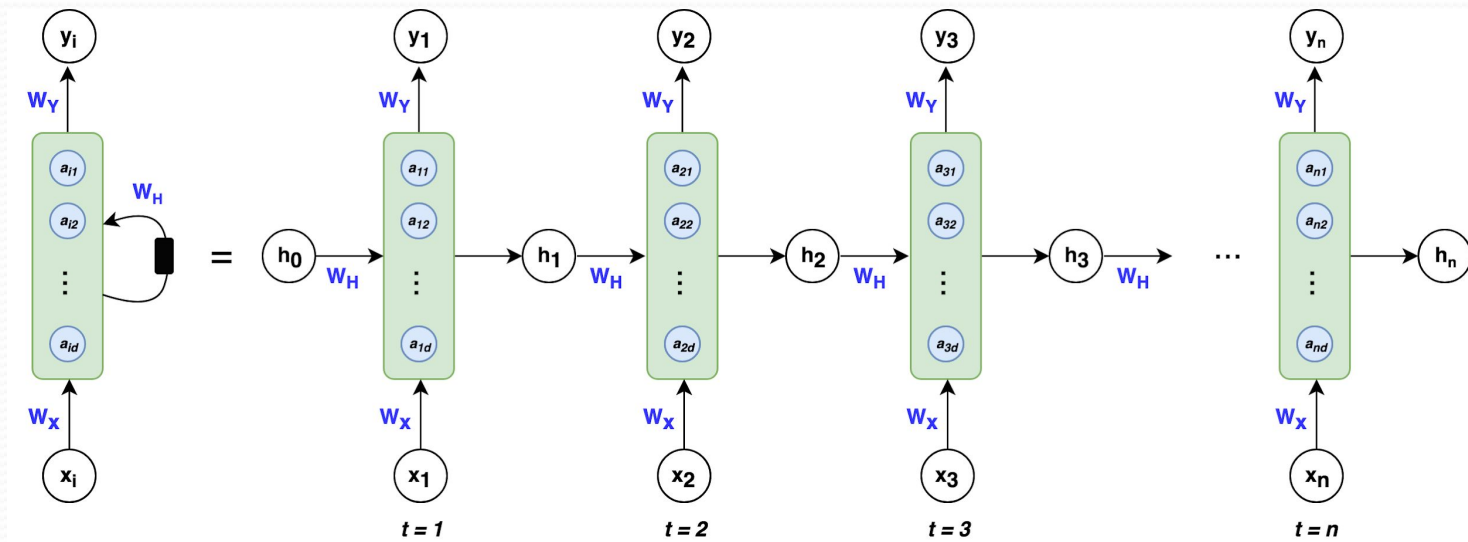
Complex Data

- Use deep learning when data is:
 - **Text** (tweets, reviews, documents)
 - **Images** (medical images, face recognition)
 - **Audio/Video** (speech recognition, surveillance)
 - Traditional ML works better for structured/tabular data.

Sequential Data

- Text
- Speech
- Time-series
- Sensor data
- Order of data matters

Recurrent neural network



- x_t : Input at time step t ($x_1, x_2, \dots, x_n$)
- h_t : Hidden state that stores past information
- h_0 : Initial hidden state
- W_x : Input $\rightarrow$ Hidden weight matrix
- W_H : Hidden $\rightarrow$ Hidden (recurrent) weight
- W_y : Hidden $\rightarrow$ Output weight matrix
- y_t : Output at time step t
- Same weights are shared across all time steps

Problems with RNN

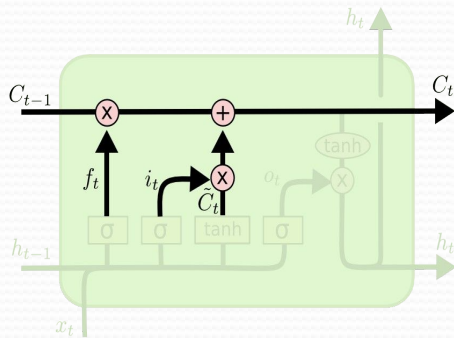
- Gradients are those values which to update neural networks weights. In other words, we can say that Gradient carries information.
- Vanishing gradient is a big problem in deep neural networks. it vanishes or explodes quickly in earlier layers and this makes RNN unable to hold information of longer sequence. and thus RNN becomes short-term memory.
- If we apply RNN for a paragraph RNN may leave out necessary information due to gradient problems and not be able to carry information from the initial time step to later time steps.

How to solve these problems?

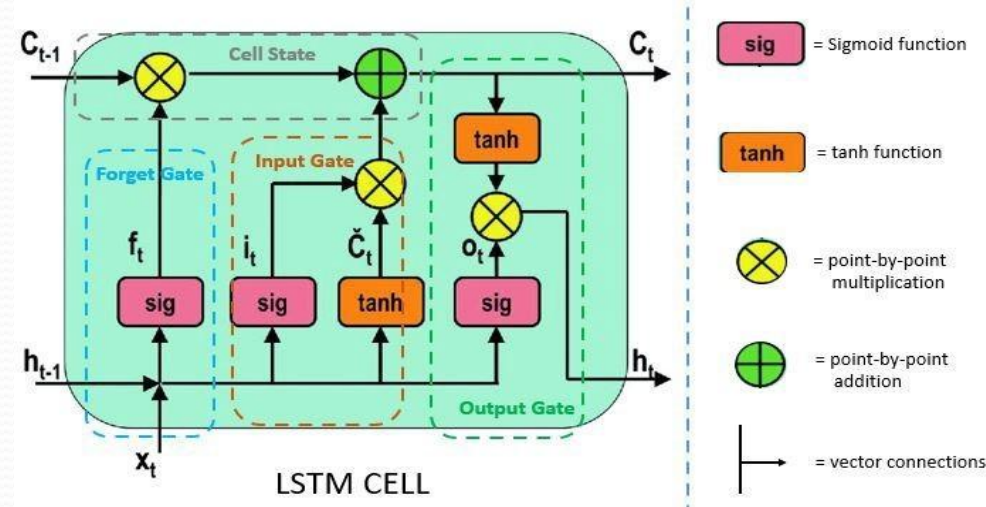
- The reason for exploding gradient was the capturing of relevant and irrelevant information a model which can decide what information from a paragraph and relevant and remember only relevant information and throw all the irrelevant information.
- This is achieved by using gates. the LSTM (Long -short-term memory) have gates as an internal mechanism, which control what information to keep and what information to throw out. By doing this LSTM networks solve the exploding and vanishing gradient problem.

LSTM

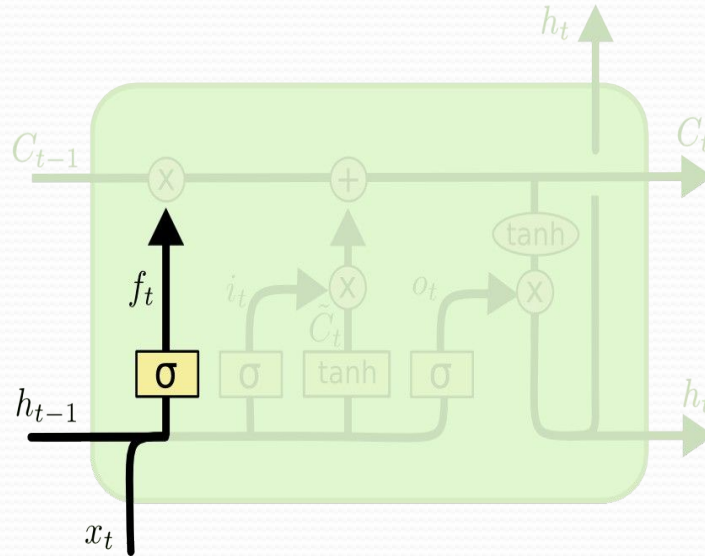
- Every LSTM network basically contains three gates to control the flow of information and cells to hold information. The Cell States carries the information from initial to later time steps without getting vanished.



$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$



Forget Gate Layer

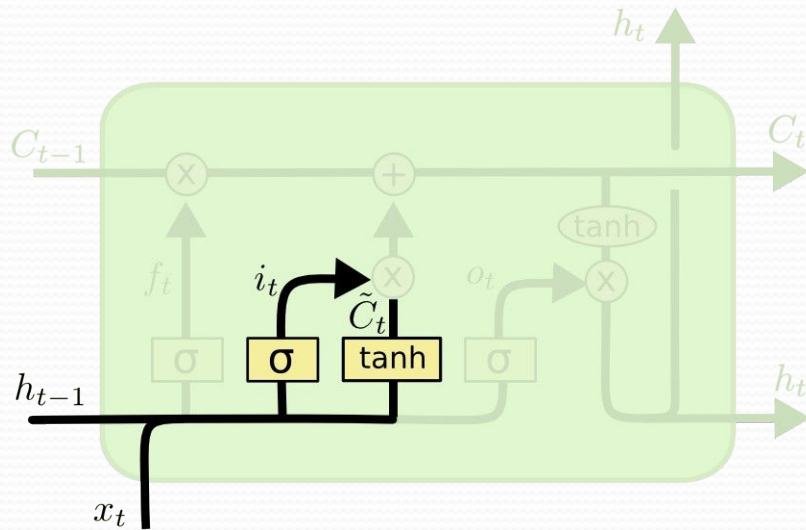


$$f_t = \sigma (W_f \cdot [h_{t-1}, x_t] + b_f)$$

Forget Gate:

- This gate decides what information should be carried out forward or what information should be ignored.
- Information from previous hidden states and the current state information passes through the sigmoid function. Values that come out from sigmoid are always between 0 and 1.
- if the value is closer to 1 means information should proceed forward and if value closer to 0 means information should be ignored.

Input Gate Layer

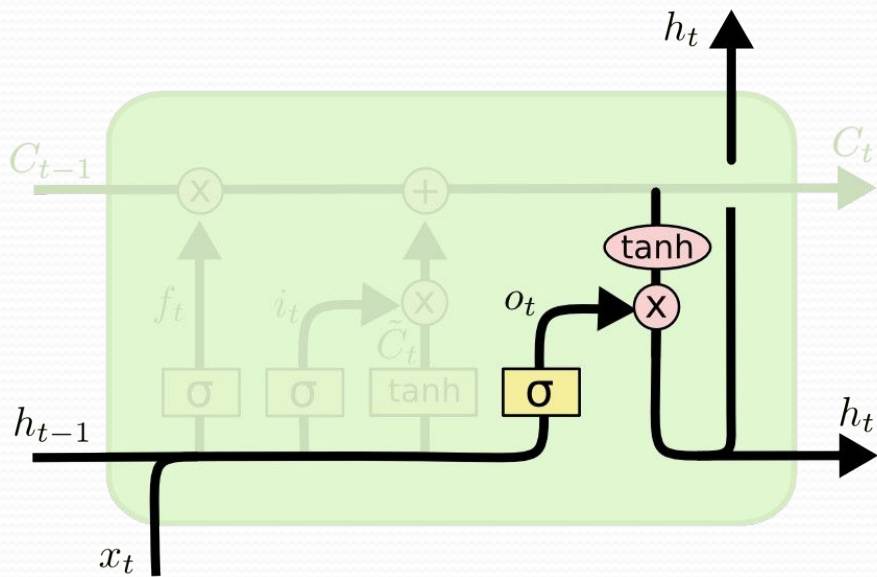


$$i_t = \sigma (W_i \cdot [h_{t-1}, x_t] + b_i)$$
$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

Input Gate:

- After deciding the relevant information, the information goes to the input gate, Input gate passes the relevant information, and this leads to updating the cell states. simply saving updating the weight.
- Input gate adds the new relevant information to the existing information by updating cell states.

Output Gate Layer



$$o_t = \sigma (W_o [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t * \tanh (C_t)$$

Output Gate:

- After the information is passed through the input gate, now the output gate comes into play.
- Output gate generates the next hidden states. and cell states are carried over the next time step.

Example: Stock prediction using LSTM

- Google stock data from 03 jan 2012 to 2017

	A	B	C	D	E	F
1	Date	Open	High	Low	Close	Volume
2	01-03-2012	325.25	332.83	324.97	663.59	73,80,500
3	01-04-2012	331.27	333.87	329.08	666.45	57,49,400
4	01-05-2012	329.83	330.75	326.89	657.21	65,90,300
5	01-06-2012	328.34	328.77	323.68	648.24	54,05,900
6	01-09-2012	322.04	322.29	309.46	620.76	116,88,800
7	01-10-2012	313.7	315.72	307.3	621.43	88,24,000
8	01-11-2012	310.59	313.52	309.4	624.25	48,17,800
9	01-12-2012	314.43	315.26	312.08	627.92	37,64,400
10	1/13/2012	311.96	312.3	309.37	623.28	46,31,800
11	1/17/2012	314.81	314.81	311.67	626.86	38,32,800
12	1/18/2012	312.14	315.82	309.9	631.18	55,44,000
13	1/19/2012	319.3	319.3	314.55	637.82	126,57,800
14	1/20/2012	294.16	294.4	289.76	584.39	212,31,800
15	1/23/2012	291.91	293.23	290.49	583.92	68,51,300
16	1/24/2012	292.07	292.74	287.92	579.34	61,34,400
17	1/25/2012	287.68	288.27	282.13	567.93	100,12,700
18	1/26/2012	284.92	286.17	281.22	566.54	64,76,500
19	1/27/2012	284.32	289.08	283.6	578.39	72,62,000
20	1/30/2012	287.95	288.92	285.63	576.11	46,78,400

	A	B	C	D	E	F
1	Date	Open	High	Low	Close	Volume
2	01-03-2017	778.81	789.63	775.8	786.14	16,57,300
3	01-04-2017	788.36	791.34	783.16	786.9	10,73,000
4	01-05-2017	786.08	794.48	785.02	794.02	13,35,200
5	01-06-2017	795.26	807.9	792.2	806.15	16,40,200
6	01-09-2017	806.4	809.97	802.83	806.65	12,72,400
7	01-10-2017	807.86	809.13	803.51	804.79	11,76,800
8	01-11-2017	805	808.15	801.37	807.91	10,65,900
9	01-12-2017	807.14	807.39	799.17	806.36	13,53,100
10	1/13/2017	807.48	811.22	806.69	807.88	10,99,200
11	1/17/2017	807.08	807.14	800.37	804.61	13,62,100
12	1/18/2017	805.81	806.21	800.99	806.07	12,94,400
13	1/19/2017	805.12	809.48	801.8	802.17	9,19,300
14	1/20/2017	806.91	806.91	801.69	805.02	16,70,000
15	1/23/2017	807.25	820.87	803.74	819.31	19,63,600
16	1/24/2017	822.3	825.9	817.82	823.87	14,74,000
17	1/25/2017	829.62	835.77	825.06	835.67	14,94,500
18	1/26/2017	837.81	838	827.01	832.15	29,73,900
19	1/27/2017	834.71	841.95	820.44	823.31	29,65,800
20	1/30/2017	814.66	815.84	799.8	802.32	32,46,600
21	1/31/2017	796.86	801.25	790.52	796.79	21,60,600
22						
23						

Data Pre processing

● Importing the data

```
7 # Importing the libraries
8 import numpy as np
9 import matplotlib.pyplot as plt
10 import pandas as pd
11
12 # Importing the training set
13 dataset_train = pd.read_csv('C:\\Users\\shiv\\Desktop\\nptel\\stock\\Stock-Price-Prediction\\
14 training_set = dataset_train.iloc[:, 1:2].values
15
```

dataset_train - DataFrame

Index	Date	Open	High	Low	Close
0	1/3/2012	325.25	332.83	324.97	663.59
1	1/4/2012	331.27	333.87	329.08	666.45
2	1/5/2012	329.83	330.75	326.89	657.21
3	1/6/2012	328.34	328.77	323.68	648.24
4	1/9/2012	322.04	322.29	309.46	620.76
5	1/10/2012	313.7	315.72	307.3	621.43
6	1/11/2012	310.59	313.52	309.4	624.25
7	1/12/2012	314.43	315.26	312.08	627.92
8	1/13/2012	311.96	312.3	309.37	623.28
9	1/17/2012	314.81	314.81	311.67	626.86
10	1/18/2012	312.14	315.82	309.9	631.18
11	1/19/2012	319.3	319.3	314.55	637.82
12	1/20/2012	294.16	294.4	289.76	584.39
13	1/23/2012	291.91	293.23	290.49	583.92

Format Resize ☒ Background color ☒ Column min/max Save and Close Close

training_set - NumPy array

	0
0	325.25
1	331.27
2	329.83
3	328.34
4	322.04
5	313.7
6	310.59
7	314.43
8	311.96
9	314.81
10	312.14
11	319.3
12	294.16

Format Resize ☒ Background color Save and Close Close

● Feature scaling

```
16 # Feature Scaling
17 from sklearn.preprocessing import MinMaxScaler
18 sc = MinMaxScaler(feature_range = (0, 1))
19 training_set_scaled = sc.fit_transform(training_set)
20
```

training_set_scaled - NumPy array

	0
0	0.0858137
1	0.0970124
2	0.0943337
3	0.0915619
4	0.0798423
5	0.0643277
6	0.0585423
7	0.0656857
8	0.0610909
9	0.0663926
10	0.0614257
11	0.0747451
12	0.0279783

Format Resize ☒ Background color

Save and Close Close

● Creating structure of data

```
20
21 # Creating a data structure with 60 timesteps and 1 output
22 X_train = []
23 y_train = []
24 for i in range(60, 1258):
25     X_train.append(training_set_scaled[i-60:i, 0])
26     y_train.append(training_set_scaled[i, 0])
27 X_train, y_train = np.array(X_train), np.array(y_train)
28
```

X_train - NumPy array

	0	1	2	3	4
0	0.0858137	0.0970124	0.0943337	0.0915619	0.0798423
1	0.0970124	0.0943337	0.0915619	0.0798423	0.0643277
2	0.0943337	0.0915619	0.0798423	0.0643277	0.0585423
3	0.0915619	0.0798423	0.0643277	0.0585423	0.0656857
4	0.0798423	0.0643277	0.0585423	0.0656857	0.0610909
5	0.0643277	0.0585423	0.0656857	0.0610909	0.0663926
6	0.0585423	0.0656857	0.0610909	0.0663926	0.0614257
7	0.0656857	0.0610909	0.0663926	0.0614257	0.0747451
8	0.0610909	0.0663926	0.0614257	0.0747451	0.0279783
9	0.0663926	0.0614257	0.0747451	0.0279783	0.0237927
10	0.0614257	0.0747451	0.0279783	0.0237927	0.0240903
11	0.0747451	0.0279783	0.0237927	0.0240903	0.0159238

Axis: 2 ▾ Shape: (1198, 60, 1) Index: 0 ▾ Slicing: [:, :, 0] Save and Close Close

X_train - NumPy array

	55	56	57	58	59
0	0.0724384	0.0799353	0.0784657	0.0803445	0.0849766
1	0.0799353	0.0784657	0.0803445	0.0849766	0.0862787
2	0.0784657	0.0803445	0.0849766	0.0862787	0.0847161
3	0.0803445	0.0849766	0.0862787	0.0847161	0.0745405
4	0.0849766	0.0862787	0.0847161	0.0745405	0.0788377
5	0.0862787	0.0847161	0.0745405	0.0788377	0.0723826
6	0.0847161	0.0745405	0.0788377	0.0723826	0.0666344
7	0.0745405	0.0788377	0.0723826	0.0666344	0.0631557
8	0.0788377	0.0723826	0.0666344	0.0631557	0.067825
9	0.0723826	0.0666344	0.0631557	0.067825	0.0682342
10	0.0666344	0.0631557	0.067825	0.0682342	0.0760101
11	0.0631557	0.067825	0.0682342	0.0760101	0.0808282

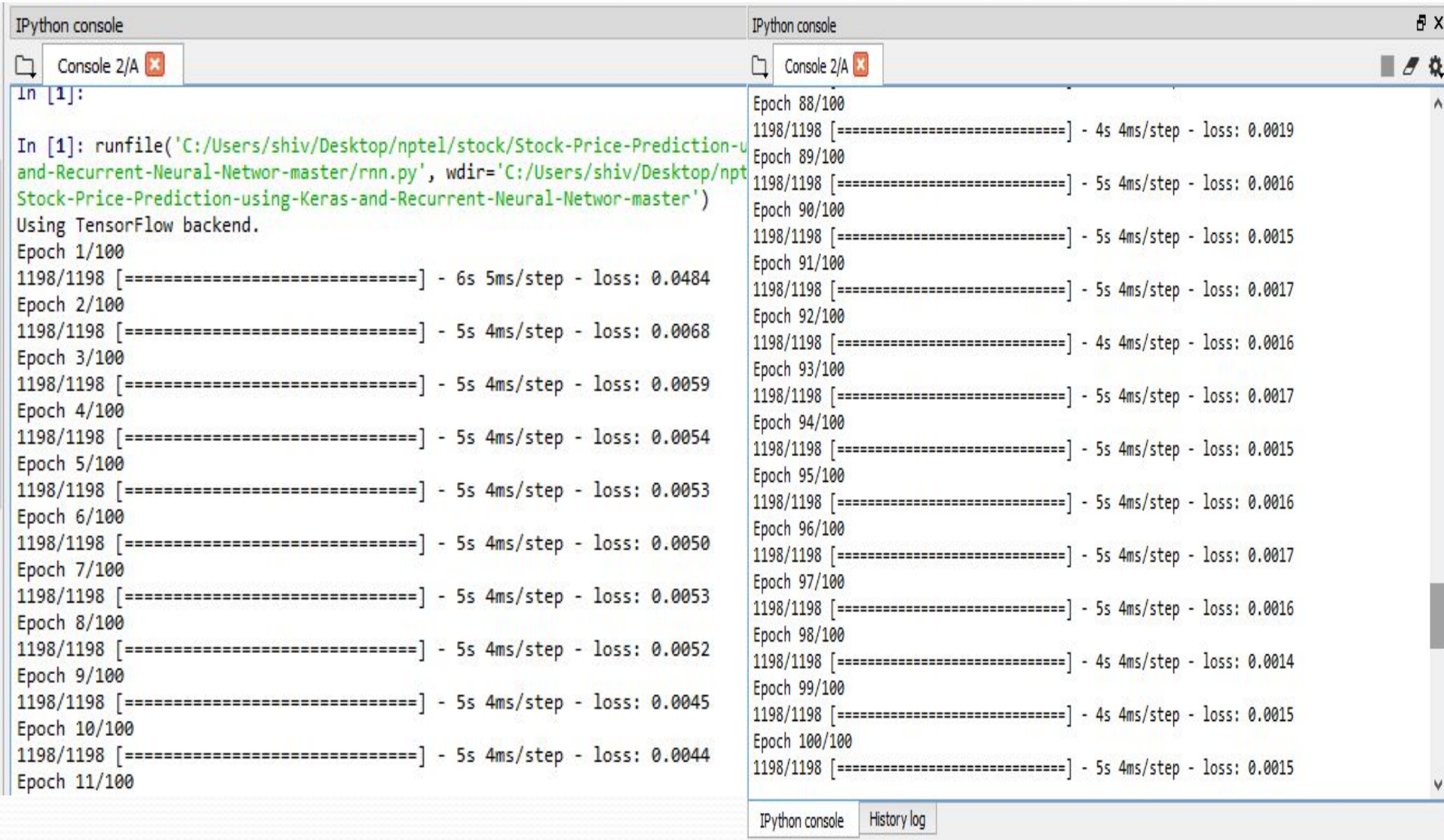
Format Resize Background color

	0
0	0.0862787
1	0.0847161
2	0.0745405
3	0.0788377
4	0.0723826
5	0.0666344
6	0.0631557
7	0.067825
8	0.0682342
9	0.0760101
10	0.0808282
11	0.0580772
12	0.0416834

Building the RNN With LSTM

```
36 # Importing the Keras libraries and packages
37 from keras.models import Sequential
38 from keras.layers import Dense
39 from keras.layers import LSTM
40 from keras.layers import Dropout
41
42 # Initialising the RNN
43 regressor = Sequential()
44
45 # Adding the first LSTM layer and some Dropout regularisation
46 regressor.add(LSTM(units = 50, return_sequences = True, input_shape = (X_train.shape[1], 1)))
47 regressor.add(Dropout(0.2))
48
49 # Adding a second LSTM layer and some Dropout regularisation
50 regressor.add(LSTM(units = 50, return_sequences = True))
51 regressor.add(Dropout(0.2))
52
53 # Adding a third LSTM layer and some Dropout regularisation
54 regressor.add(LSTM(units = 50, return_sequences = True))
55 regressor.add(Dropout(0.2))
56
57 # Adding a fourth LSTM layer and some Dropout regularisation
58 regressor.add(LSTM(units = 50))
59 regressor.add(Dropout(0.2))
60
61 # Adding the output layer
62 regressor.add(Dense(units = 1))
63
64 # Compiling the RNN
65 regressor.compile(optimizer = 'adam', loss = 'mean_squared_error')
66
67 # Fitting the RNN to the Training set
68 regressor.fit(X_train, y_train, epochs = 100, batch_size = 32)
69
70
71
```

- At every Epoch LOSS is get reduced



```
IPython console
Console 2/A x

In [1]:
In [1]: runfile('C:/Users/shiv/Desktop/nptel/stock/Stock-Price-Prediction-using-Keras-and-Recurrent-Neural-Network-master/rnn.py', wdir='C:/Users/shiv/Desktop/nptel/stock/Stock-Price-Prediction-using-Keras-and-Recurrent-Neural-Network-master')
Using TensorFlow backend.
Epoch 1/100
1198/1198 [=====] - 6s 5ms/step - loss: 0.0484
Epoch 2/100
1198/1198 [=====] - 5s 4ms/step - loss: 0.0068
Epoch 3/100
1198/1198 [=====] - 5s 4ms/step - loss: 0.0059
Epoch 4/100
1198/1198 [=====] - 5s 4ms/step - loss: 0.0054
Epoch 5/100
1198/1198 [=====] - 5s 4ms/step - loss: 0.0053
Epoch 6/100
1198/1198 [=====] - 5s 4ms/step - loss: 0.0050
Epoch 7/100
1198/1198 [=====] - 5s 4ms/step - loss: 0.0053
Epoch 8/100
1198/1198 [=====] - 5s 4ms/step - loss: 0.0052
Epoch 9/100
1198/1198 [=====] - 5s 4ms/step - loss: 0.0045
Epoch 10/100
1198/1198 [=====] - 5s 4ms/step - loss: 0.0044
Epoch 11/100

IPython console
Console 2/A x

Epoch 88/100
1198/1198 [=====] - 4s 4ms/step - loss: 0.0019
Epoch 89/100
1198/1198 [=====] - 5s 4ms/step - loss: 0.0016
Epoch 90/100
1198/1198 [=====] - 5s 4ms/step - loss: 0.0015
Epoch 91/100
1198/1198 [=====] - 5s 4ms/step - loss: 0.0017
Epoch 92/100
1198/1198 [=====] - 4s 4ms/step - loss: 0.0016
Epoch 93/100
1198/1198 [=====] - 5s 4ms/step - loss: 0.0017
Epoch 94/100
1198/1198 [=====] - 5s 4ms/step - loss: 0.0015
Epoch 95/100
1198/1198 [=====] - 5s 4ms/step - loss: 0.0016
Epoch 96/100
1198/1198 [=====] - 5s 4ms/step - loss: 0.0017
Epoch 97/100
1198/1198 [=====] - 5s 4ms/step - loss: 0.0016
Epoch 98/100
1198/1198 [=====] - 4s 4ms/step - loss: 0.0014
Epoch 99/100
1198/1198 [=====] - 4s 4ms/step - loss: 0.0015
Epoch 100/100
1198/1198 [=====] - 5s 4ms/step - loss: 0.0015
```

prediction on Test Set

```
88 predicted_stock_price = regressor.predict(X_test)
89 predicted_stock_price = sc.inverse_transform(predicted_stock_price)
90
91 # Visualising the results
92 plt.plot(real_stock_price, color = 'red', label = 'Real Google Stock Price')
93 plt.plot(predicted_stock_price, color = 'blue', label = 'Predicted Google Stock Price')
94 plt.title('Google Stock Price Prediction')
95 plt.xlabel('Time')
96 plt.ylabel('Google Stock Price')
97 plt.legend()
98 plt.show()
```

predicted_stock_price - NumPy array

	0
0	769.418
1	766.91
2	766.648
3	767.676
4	770.6
5	775.94
6	781.29
7	784.014
8	784.634
9	784.231
10	783.583
11	782.948
12	782.523

Format

Resize

☒ Background color

Save and Close

Close

real_stock_price - NumPy array

	0
0	778.81
1	788.36
2	786.08
3	795.26
4	806.4
5	807.86
6	805
7	807.14
8	807.48
9	807.08
10	805.81
11	805.12
12	806.91

Format

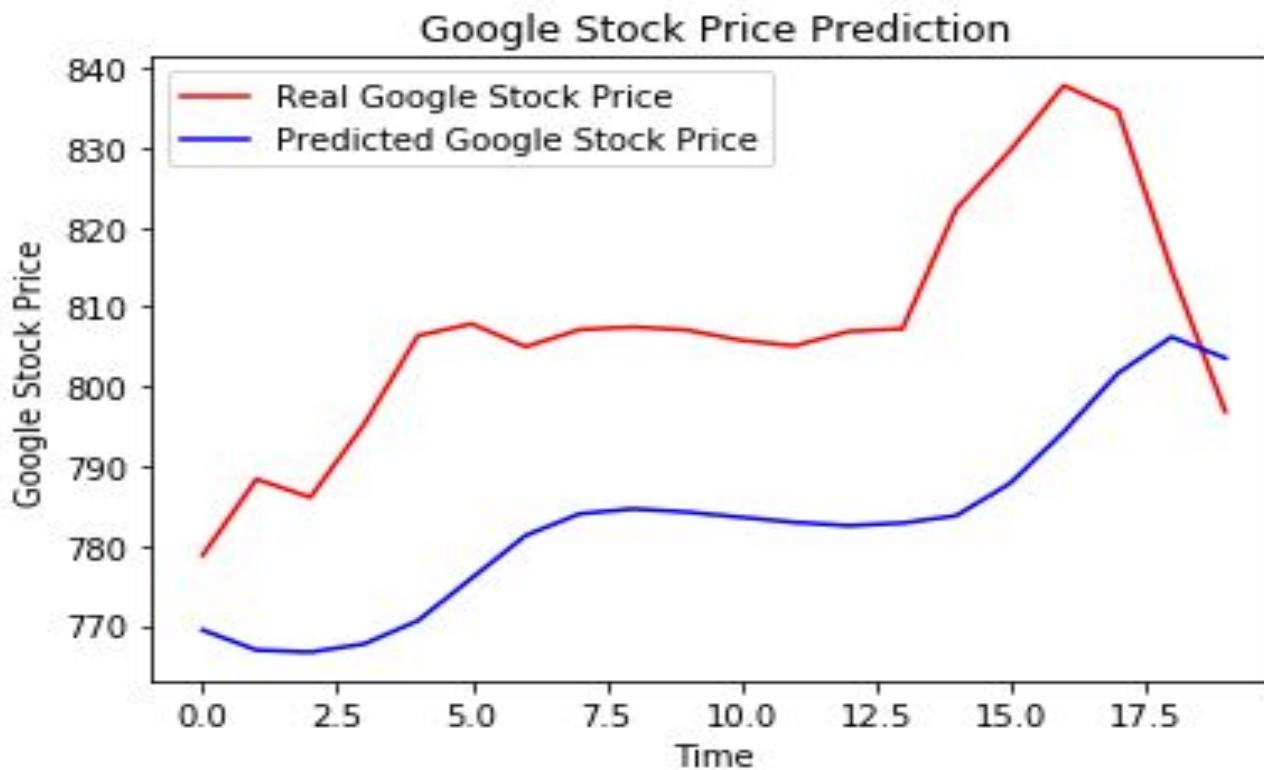
Resize

☒ Background color

Save and Close

Close

Graph of real vs predicted



References

- Al Ajlouni, Ahmed & Al - Hakim, Monir Suliaman. (2019). Financial Technology in Banking Industry: Challenges and Opportunities.
- Siau, Keng & Hilgers, Mike & Chen, Langtao & Liu, Yu & Nah, Fiona & Hall, Richard & flachsbart, Barry. (2018). FinTech Empowerment: Data Science, Artificial Intelligence, and Machine Learning. 31. 12-18.
- Bazarbash, Majid. (2019). FinTech in Financial Inclusion Machine Learning Applications in Assessing Credit Risk. 10.5089/9781498314428.001.
- Fuster, A., P. Goldsmith-Pinkham, T. Ramadorai, and A. Walther (2018). “Predictably unequal?,” The Effects of Machine Learning on Credit Markets. November 6.
- Gonçalo Duarte Lima Freire Lopes. Deep learning for market forecasts. 2018.



● THANK YOU